SNES Assembly Adventure 04: Display Your First Sprite

21 Sep 2018

Update February 2022: All code examples from all articles in this series can now be found on Github in one repository

Welcome back, Adventurer. I apologize for the long wait between articles. I got caught up in other things. But now we're back!

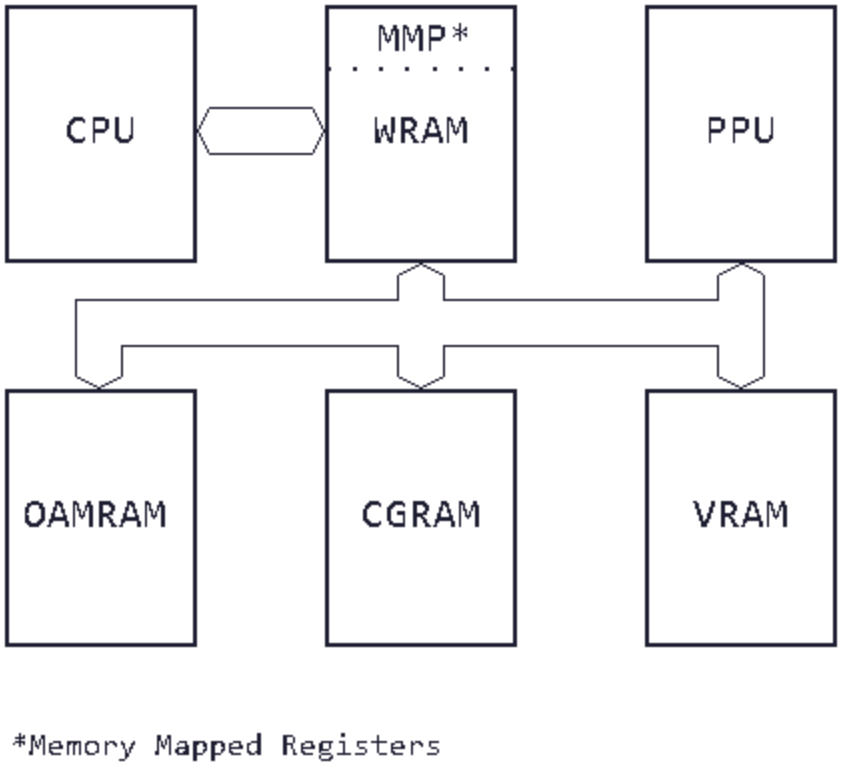
Last time, we created a simple sprite. In this article, we will take a closer look at the SNES architecture and memory mapped registers. And then finally display the sprite from last time.



Like *Lethal Weapon IV*, just better

Displaying a Sprite

We will discuss the architecture of the SNES in the next articles in more detail. For now, look at this simplified overview:



The simplified SNES Architecture

Here is a summary of all components. The CPU or *Central Processing Unit* is the main processor of the SNES, this is the 65816 that will run the code we write. The PPU or *Picture Processing Unit* is responsible for rendering and sending data to the TV screen. The PPU has its own separate RAM that we cannot access directly.

As shown in the picture above, the SNES has four distinct sections of RAM:

- **WRAM**: Work RAM. This is the general RAM we can fully access and use for pretty much anything we want; graphics, variables, tables, etc.
- **VRAM**: Video RAM. Here is where we will store the graphics data we want to display. We will place the sprites we created last time here.
- **CGRAM**: Color Generator RAM. This is a special portion of RAM that will hold the actual color data.

- **OAMRAM**: Object Attribute Memory. Another special portion of RAM. Here we store the information to tell the PPU *where* to draw *which* sprite. We return to OAM in a moment.

(I left out the audio portion on purpose. I will cover audio in a later article.)

So, for the SNES to display our sprites we need to move the sprite data to VRAM, and the palette data to CGRAM. How do we do that? Memory Mapped Registers is the answer.

Memory Mapped Registers

When utilizing this technique, we reserve certain portions of the memory map (i.e., a range of memory addresses) for special purposes. In the case of the SNES, the addresses $xx:2100 through $xx:2143 are reserved to send commands and data to the PPU.

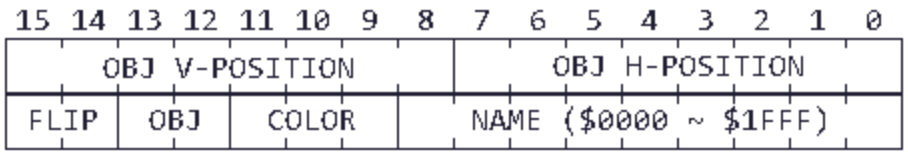
In the *Links and References* section at the end of this article, you'll find a link to a list of all Memory Mapped Registers of the SNES. Use this list while you work through the code example below to understand the commands we send to the PPU.

This is also the only way for us to access VRAM, CGRAM, and OAMRAM. We tell the PPU that we want to change the data in one of these memory sections. Then send the (new) data we want to be changed/written to PPU RAM. Simple as that.

Object Attribute Memory

OAM or *Object Attribute Memory* is a section within PPU RAM reserved for a special purpose. Once we have the graphics and color data (from `Sprites.vra` and `SpriteColors.pal`) loaded into VRAM, we also need to tell the PPU *where* to draw *which* sprites on the screen.

This is what OAM does for us. Every sprite we want to display needs four bytes of information in OAM:



The simplified SNES Architecture

These parameters control how a sprite is displayed/rendered by the PPU:

- **H-Position**: The horizontal position in pixels

- **V-Position**: The vertical position in pixels
- **Name**: Which sprite from the sprite sheet to display
- **Color**: A 3-bit number, select which out of eight color palettes to use
- **Obj**: Set the priority relative to backgrounds
- **Flip**: Whether to flip the sprite horizontally and/or vertically

We'll discuss *Obj* and *Flip* in more detail in a later article. For now, we won't use them/set them to zero.

That should be enough to understand the demo of this article. Let's see all this in action and make some sense of it.

The Source Code

Okay, finally we get down to business. Here is what the code example will do:

- Load `Sprites.vra` into VRAM
- Load `SpriteColors.pal` into CGRAM
- Load OAM data into OAMRAM
- Get stuck in an endless loop

Please keep two things in mind. First, I'll introduce you to a few new opcodes. We'll not discuss them in full detail yet, because most of them are just variants of opcodes you already know (can you guess what LDX does?). But I will discuss them in more detail in the next articles. Second, this is definitely *not* the way you want to do this. There are more effective ways to move data to (and from) PPU RAM. This example's focus is to understand how memory mapped registers work.

Let's look at the code: